

```

# p@strings:~
GPT Data 2.2.6 File: led.c
#include <stdio.h>
#include <string.h>

#define LED 7

int main ("0x0")
{
    printf("MapMemory Pa blank/a") ;

    wiringPiSetup () ;
    pinMode (LED, OUTPUT) ;

    for (i=0;
    {
        printf("LED ON") ;
        digitalWrite (LED, HIGH) ; // On
        delay (500) ; // ms
        printf("LED OFF") ;
        digitalWrite (LED, LOW) ; // Off
        delay (500) ;
    }
    return 0;
}

```

```
pi@raspberrypi ~$ sudo nano led.c
pi@raspberrypi ~$ gcc -o blink led.c -lwiringPi
```

```
pi@raspberrypi ~$ sudo nano led.c
pi@raspberrypi ~$ gcc -o blink led.c -lwiringPi
pi@raspberrypi ~$ sudo ./blink
```

[illegible]

Pi. To download and install the library follow guidelines on <http://wiringpi.com/download-and-install/>

Once the library is installed, we are all set to write a C program on Raspberry Pi. Before writing it, one needs to understand how to address/refer GPIO pins.

Referring to pins on the expansion header

Any signal (or pin) can be referred to in program in three ways:

1. Broadcom signal names on the SoC (BCM GPIO NUMBER), for example, GPIO 4

2. Pin names on the Raspberry Pi board headers (PIN NUMBER), for example, PIN 7 or PI-07

3. Names used in the WiringPi library (WIRING_PI_PIN), for example, 7

Table I shows different names for each Raspberry pin.

In this article, we are going to use third scheme (WiringPi pin numbers, that column from Table I).

Project 1: Blinking the LED using Wiring Pi C

In this project, we will access the GPIO pin of Raspberry Pi and control it for blinking an LED using the WiringPi C. Connect the LED to pin number 7 (BCM GPIO 4) of Raspberry Pi as shown in Fig. 2.

Software

Start with C programming and open LxTerminal to create new C file by using the following command:

```
$ touch led.c
```

Then start nano text editor as:

Type the following C code or copy and paste it to the file as shown in Fig. 3:

```
#include <stdio.h>
#include <wiringPi.h>

#define LED 7

int main (void)
{
    printf ("Raspberry Pi blink\n") ;

    wiringPiSetup () ;
    pinMode (LED, OUTPUT) ;

    for (;;)
    {
        printf ("LED ON");
        digitalWrite (LED, HIGH) ; // On
        delay (500) ; // mS
        printf ("LED OFF");
        digitalWrite (LED, LOW) ; // Off
        delay (500) ;
    }

    return 0;
}
```

Once you are finished, press Ctrl + O to save and then Ctrl + X to exit from the nano editor.

Now, compile the program by typing the following command as shown in Fig. 4

```
$ gcc -o blink led.c -lwiringPi
```

In the above compilation command, the word after -o denotes name of output file. At the end, we need to add -lwiringPi to notify the compiler that we are using library WiringPi (note, only P is capital in the last word).

Now run the program using the following command as shown in Fig. 5:

```
$ sudo ./blink
```

In the above execution command, dot slash is followed by output filename that we specified in the compilation command. After running the program, LED will start blinking and the LED ON and LED OFF will be displayed on the terminal of Raspberry Pi as shown in Fig. 6.

You can observe the output on screen and see LED blinking according to the program. You can also change the delay and Raspberry Pi pin as per your needs.

Functions of WiringPi library used in led.c

WiringPiSetup(void). This initialises